BÀI TẬP LÝ THUYẾT
LẬP KẾ HOẠCH VÀ ĐỒ THỊ KẾ HOẠCH TRONG AI

Họ và tên	**Trương Văn Thiện**
MSSV / Lớp	**3123410356**

Mục tiêu học tập

- Mô tả bài toán lập kế hoạch cổ điển bằng trạng thái, mục tiêu, toán tử và các tập Precondition/Add/Delete.
- Xây dựng planning graph theo GraphPlan với các mức trạng thái S_i, mức hành động A_i và các hành động No-op.
- Nhận diện và giải thích các quan hệ mutex: Inconsistent Effects, Interference và Competing Needs.
- Phân tích điều kiện dừng của GraphPlan và trích xuất kế hoạch từ planning graph.
- Vận dụng GraphPlan vào các bài toán Crane, Valet Parking và môi trường planning rời rạc.

Hướng dẫn: Trả lời bằng tiếng Việt, trình bày rõ các bước suy luận. Với câu có planning graph, có thể vẽ trực tiếp lên hình hoặc bổ sung bảng. No-op được tính là một hành động khi xét mutex.

Bài 1. Khái niệm cơ bản trong GraphPlan

Hãy định nghĩa ngắn gọn các khái niệm sau và nêu ý nghĩa của chúng trong việc xây dựng planning graph.

a) Interference (xung đột do can thiệp):

Hai hành động mutex nếu hiệu ứng của hành động này phá tiền điều kiện của hành động kia.
Ý nghĩa: loại các hành động không thể thực hiện đồng thời.

b) Inconsistent Effects / Inconsistency (hiệu ứng không nhất quán):

Hai hành động có hiệu ứng đối nghịch: một thêm p, hành động kia thêm ¬p/xóa p.
Ý nghĩa: ngăn việc tạo các kết quả mâu thuẫn trong cùng một mức.

c) Competing Needs (tiền điều kiện cạnh tranh):

Tiền điều kiện của hai hành động mutex ở mức trạng thái liền trước.
Ý nghĩa: chúng không thể cùng thực hiện trong một bước.

d) Có những cách nào để tìm một kế hoạch trong mô hình lập kế hoạch cổ điển?

Tìm kiếm tiến từ trạng thái đầu; tìm kiếm lùi từ trạng thái mục tiêu.
Lập kế hoạch thứ tự bộ phận (POP); tìm kiếm heuristic (A*, greedy).
GraphPlan truy vết đồ thị, hoặc biến đổi bài toán sang SAT/CSP.

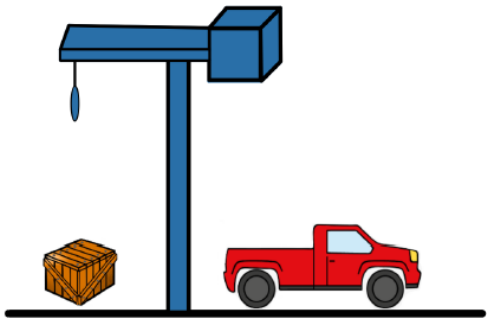
e) Khi dùng heuristic dựa trên GraphPlan, những giả định nào của classical planning được nới lỏng? Vì sao điều này hữu ích hơn so với tìm kiếm ngây thơ?

Relaxed planning graph thường bỏ hiệu ứng xóa và/hoặc bỏ mutex.
Nhờ đó ước lượng khoảng cách tới mục tiêu nhanh hơn.
Heuristic dẫn hướng tìm kiếm, tránh duyệt mù quá nhiều trạng thái.

Bài 2. Lập kế hoạch ký hiệu - Bài toán cần cẩu

Cho một cần cẩu, một kiện hàng và một xe tải. Ban đầu kiện hàng ở bên trái, xe tải ở bên phải và cần cẩu quay về bên trái. Mục tiêu là đưa kiện hàng lên xe tải và cuối cùng cần cẩu quay về bên trái.

Cần cẩu có thể quay giữa trái và phải, có hoặc không mang kiện hàng. Cần cẩu có thể nhấc kiện hàng nếu cả hai ở cùng phía. Kiện hàng chỉ có thể được đưa lên xe tải bằng cần cẩu.



a) Xác định các biến/đối tượng, các predicate cần dùng, trạng thái ban đầu và trạng thái mục tiêu.

Đối tượng: cần cẩu, kiện hàng, xe tải; hai phía trái (L), phải (R).
Predicate: FL, FR (hướng cẩu); L (kiện trái); H (đang giữ);
E (tay rỗng); T (kiện trên xe). S_0={FL,L,E}; G={T,FL}.

b) Xây dựng các toán tử hành động. Với mỗi toán tử, ghi rõ Preconditions, Add List và Delete List.

Nhấc P: Pre={FL,L,E}; Add={H}; Del={L,E}.
Quay LR0: Pre={FL,E}; Add={FR}; Del={FL}.
Quay RL0: Pre={FR,E}; Add={FL}; Del={FR}.
Quay LRH/RLH: Pre={FL,H}/{FR,H}; Add={FR}/{FL}; Del={FL}/{FR}.
Đặt D: Pre={FR,H}; Add={T,E}; Del={H}. No-op(X): Pre/Add={X}.

c) Vẽ planning graph cho 3 bước đầu tiên. Có thể dùng hình minh họa thay cho proposition nếu biểu diễn rõ ràng.

S_0={FL,L,E}; A_0={P,LR0,N(FL),N(L),N(E)}.
S_1={FL,FR,L,H,E}; A_1={P,LR0,RL0,LRH và các No-op}.
S_2={FL,FR,L,H,E}; A_2 có D và các hành động/No-op hợp lệ.
S_3 có T nhưng T và FL còn mutex; cần mở rộng đến S_4.
Kế hoạch: P → LRH → D → RL0; đạt G={T,FL} ở S_4.

d) Vẽ hai mức đầu tiên của GraphPlan và chỉ ra các cặp hành động mutex; với mỗi cặp, ghi loại mutex.

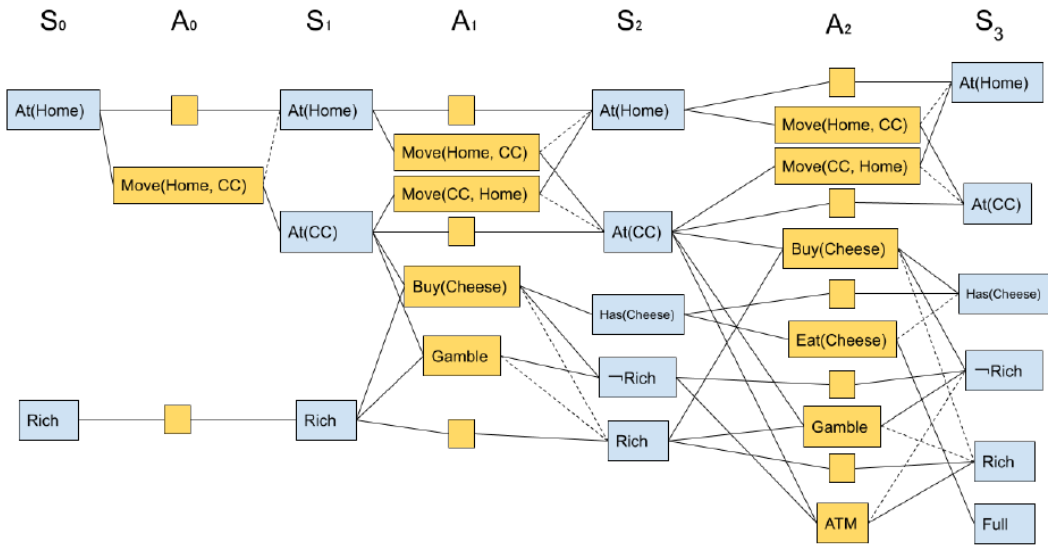
Ký hiệu Q=LR0, R=RL0, K=LRH, N(X)=No-op(X).
A_0: P–Q [I]; P–N(L/E), Q–N(FL) [IE].
A_1 [IE]: P–N(L/E), Q–R/N(FL), R–K/N(FR), K–N(FL).
A_1 [I]: P–Q/R/K, Q–K. [CN]: P,Q–N(H/FR); R–N(FL/H);
A_1 [CN]: K–N(FR/E/L); N(FL)–N(FR); N(H)–N(FR/E/L).

Bài 3. Quan hệ Mutex trong planning graph

Pinky đi lấy thức ăn. Các hành động được cho như sau:

Hành động	Preconditions	Add List	Delete List
Move(A,B)	At(A)	At(B)	At(A)
Buy(Cheese)	At(ChuckyCheese), Rich	Has(Cheese), ¬Rich	
Gamble	At(ChuckyCheese), Rich	¬Rich	Rich
ATM	At(ChuckyCheese), ¬Rich	Rich	¬Rich
Eat(Cheese)	Has(Cheese)	Full	Has(Cheese)

Trạng thái bắt đầu: Rich ∧ At(Home). Mục tiêu: Full. Planning graph tương ứng được cho trong hình dưới:



Planning graph cho bài toán Pinky.

a) Ở mức A_0, hãy chỉ ra một cặp hành động mutex do Inconsistent Effects.

Move(Home, CC) và N[At(Home)] mutex do Inconsistent Effects.
Move xóa At(Home), còn N[At(Home)] duy trì At(Home).
Cặp này ở A_0; CC viết tắt cho ChuckyCheese.

b) Ở mức A_1, hãy chỉ ra một cặp hành động mutex do Interference.

Buy(Cheese) và Gamble mutex do Interference tại A_1.
Gamble xóa Rich, là tiền điều kiện của Buy(Cheese).
Cả hai cần Pinky ở ChuckyCheese và còn Rich.

c) Ở mức A_2, hãy chỉ ra một cặp hành động mutex do Competing Needs.

N[At(Home)] và Buy(Cheese) mutex do Competing Needs tại A_2.
At(Home) và At(CC) là hai tiền điều kiện mutex ở S_2.
Pinky không thể có mặt ở hai địa điểm cùng lúc.

d) Giải thích ngắn gọn vì sao mỗi cặp ở trên thỏa loại mutex tương ứng.

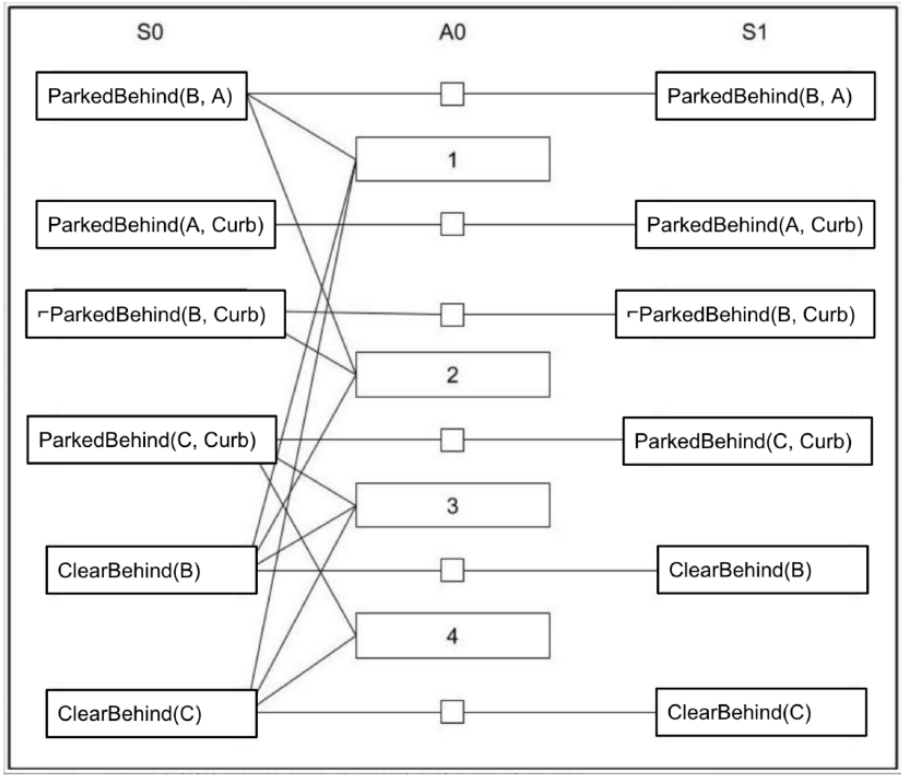
(a) Hiệu ứng giữ At(Home) và xóa At(Home) là đối nghịch.
(b) Gamble xóa Rich, phá tiền điều kiện của Buy.
(c) At(Home) và At(CC) không thể đồng thời đúng.

Bài 4. Classical Planning và GraphPlan - Valet Parking

Xét bài toán Valet Parking với hai predicate ClearBehind(car) và ParkedBehind(car1, car2). Hai toán tử được định nghĩa như sau:

Thành phần	**ParkBehind(car1, car2)**	**ParkInNewRow(car)**
Preconditions	ClearBehind(car1) ClearBehind(car2) ParkedBehind(car1, place)	ClearBehind(car) ParkedBehind(car, place) ¬ParkedBehind(car, curb)
Add List	ParkedBehind(car1, car2) ClearBehind(place) ¬ParkedBehind(car1, place) ¬ClearBehind(car2)	ParkedBehind(car, curb) ClearBehind(place) ¬ParkedBehind(car, place)
Delete List	ClearBehind(car2) ParkedBehind(car1, place)	ParkedBehind(car, place) ¬ParkedBehind(car, curb)

Hình duowis là mẫu cho hai mức đầu tiên $S_0 \rightarrow A_0 \rightarrow S_1$. Các nút hành động No-op đã được biểu diễn bằng các ô vuông nhỏ không tên.



Mẫu GraphPlan cho Valet Parking

a) Điền tên hành động phù hợp vào các ô 1-4 ở mức A_0. Nếu không tồn tại hành động hợp lệ cho tập tiền điều kiện đó, ghi "N/A".

1: ParkBehind(B,C); 2: ParkInNewRow(B).
3: ParkBehind(C,B); 4: N/A (thiếu ¬ParkedBehind(C,curb)).
Viết tắt PB(x,y)=ParkedBehind(x,y); CB(x)=ClearBehind(x).

b) Với từng hành động 1-4, liệt kê TẤT CẢ predicate ở S_1 nhận add-edge từ hành động đó. Nếu hành động không hợp lệ, ghi "N/A".

1: PB(B,C), CB(A), ¬PB(B,A), ¬CB(C).
2: PB(B,curb), CB(A), ¬PB(B,A).
3: PB(C,B), CB(curb), ¬PB(C,curb), ¬CB(B); 4: N/A.

c) Nêu hai action node có quan hệ mutex do Inconsistent Effects. No-op được phép sử dụng.

ParkBehind(B,C) và No-op[ClearBehind(C)] mutex do IE.
Nút 1 thêm ¬CB(C), còn No-op duy trì CB(C).
Hai hiệu ứng của chúng đối nghịch nhau.

d) Nêu hai action node có quan hệ mutex do Interference.

ParkBehind(B,C) và ParkBehind(C,B) mutex do Interference.
Nút 1 xóa CB(C), là tiền điều kiện nút 3; nút 3 xóa CB(B),
là tiền điều kiện nút 1.

e) GraphPlan "leveled off" khi nào? Chọn một đáp án:

☐ A. Tất cả các hành động có thể đã được khám phá.

☐ B. Không còn tập literal khác rỗng nào không có liên kết mutex.

☒ C. Hai mức liên tiếp của planning graph giống hệt nhau.

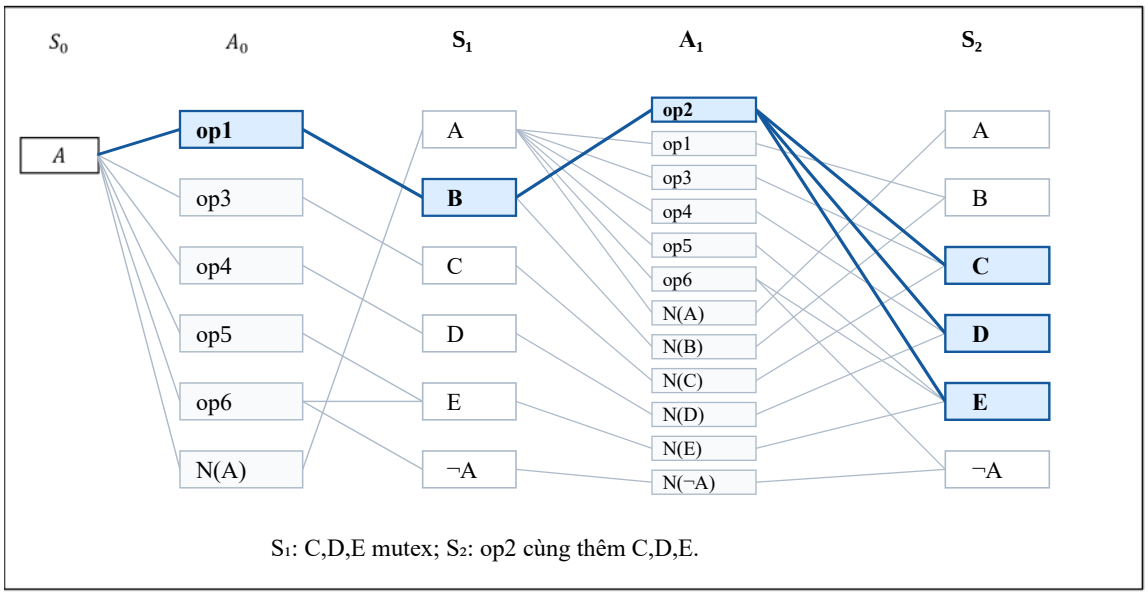
☐ D. Mức trạng thái cuối cùng chứa một trạng thái mục tiêu.

Bài 5. Xây dựng GraphPlan và trích xuất kế hoạch

Cho môi trường lập kế hoạch với 6 toán tử. Trạng thái bắt đầu là A; mục tiêu là C ∧ D ∧ E. Mỗi thời điểm chỉ áp dụng một toán tử và cần tìm kế hoạch có ít thao tác nhất.

	op1	**op2**	**op3**	**op4**	**op5**	**op6**
Precondition	A	B	A	A	A	A
Add	B	C, D, E	C	D	E	E, ¬A
Delete						

a) Chạy GraphPlan: vẽ lần lượt các mức trạng thái và mức hành động cho đến khi thuật toán dừng. Nhớ bổ sung các No-op cho trạng thái cần duy trì.



Khung để hoàn thiện planning graph; trạng thái đầu S_0 = {A}.

b) Kế hoạch nào được GraphPlan trả về?

Kế hoạch: op1 → op2, gồm 2 thao tác.
op1 tạo B; op2 cần B rồi thêm đồng thời C, D, E.
Đạt G={C,D,E}; các No-op chỉ giúp duy trì literal.

c) Kế hoạch trên có tối ưu về số lượng thao tác hay không? ☒ Có ☐ Không

Có. Không toán tử nào tạo đủ C, D, E chỉ trong 1 bước.
op1 tạo B, tiếp đó op2 tạo C, D, E cùng lúc.
Vậy 2 thao tác là tối thiểu với 1 toán tử/bước.

d) Liệt kê TẤT CẢ cặp toán tử mutex ở A_0 và TẤT CẢ cặp proposition mutex ở S_1. Nếu không có, ghi "Không có".

A_0 (serial): (1,3), (1,4), (1,5), (1,6), (3,4), (3,5),
(3,6), (4,5), (4,6), (5,6); thêm (op6,N(A)).
S_1: (A,¬A), (B,C/D/E/¬A), (C,D/E/¬A), (D,E/¬A).

Bài 6. Tổng hợp và phân tích

a) So sánh tìm kiếm trực tiếp trong không gian trạng thái với GraphPlan. Trong trường hợp nào planning graph giúp giảm đáng kể không gian tìm kiếm?

Tìm kiếm trạng thái duyệt từng trạng thái nên dễ bùng nổ nhánh.

GraphPlan gộp hành động vào các mức và loại tổ hợp mutex.
Rất hữu ích khi nhiều hành động độc lập, mục tiêu nhiều literal.

b) Vì sao No-op quan trọng trong GraphPlan? Nếu bỏ toàn bộ No-op thì việc biểu diễn trạng thái tồn tại qua nhiều mức sẽ bị ảnh hưởng như thế nào?

No-op(X): Pre={X}, Add={X}; giúp giữ X qua các mức S_i.
Nếu bỏ No-op, literal không được thêm lại sẽ biến mất,
làm mất tính liên tục và có thể bỏ sót kế hoạch đúng.

c) Cho một mục tiêu gồm nhiều literal. Hãy giải thích vai trò của mutex giữa các literal mục tiêu trong quyết định mở rộng planning graph hoặc kết luận thất bại.

Nếu các mục tiêu mutex ở S_i, chúng chưa thể đồng thời đạt.
Cần mở rộng đồ thị; không mutex thì thử trích xuất kế hoạch.
Thất bại khi đồ thị leveled off mà truy vết vẫn không thành.

d) Nêu quy trình tổng quát để trích xuất một kế hoạch từ planning graph sau khi tất cả literal mục tiêu đã xuất hiện ở cùng một mức và không mutex với nhau.

Tại S_k, chọn các hành động không mutex tạo/giữ mục tiêu.
Thay mục tiêu bằng hợp tiền điều kiện ở S_{k-1}; lặp về S_0.
Nếu bế tắc, quay lui/mở rộng; đảo chuỗi để có kế hoạch.

— Hết —